Robot Movement Planning

Constraint Programming

Alessio Russo (mat. 376856)

1 Problem Statement

We consider a stream of identical boxes arriving on a belt. Each box has size $x \times y$, where x is parallel to the belt direction and y is orthogonal to it. A target pallet capable of containing n boxes is given. Each box i has a destination position on the pallet, given by $(P_x[i], P_y[i])$, where $i = 1, \dots, n$.

The robot can pick a consecutive set of boxes from the stream. Since boxes on the belt have no gaps between them, each picked set is contiguous along the x-axis. Given a gripper size S, the total length of the picked set must be at most $S + x$. This corresponds to an overflow of at most $x/2$ on each side of the gripper. Therefore, the maximum number of boxes that can be picked at once is $\left\lfloor \frac{S+x}{x} \right\rfloor$.

After picking a set of boxes, the robot releases it on the pallet. Figure 1 shows an example of the picking and releasing operations.

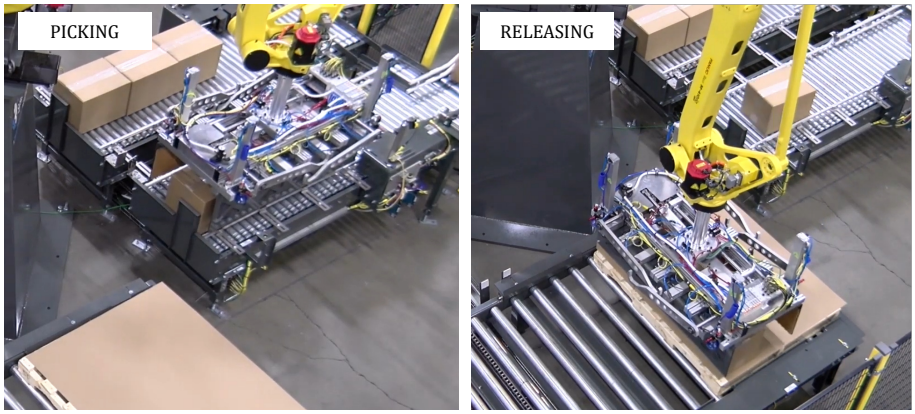


Figure 1: Example of picking and releasing operations.

The gripper has two parallel plates, but only one plate can move during release. For this reason, the opening side must be free. It can face either the outside of the pallet or a position that has not been filled yet. We split the problem into two parts:

1. **Grouping.** Identify the sets of boxes that can be picked together. Each group must contain boxes on the same row, aligned along the x-axis, and must fit within the gripper capacity.

2. **Scheduling.** Choose the order in which the groups are released. For each group, also choose which plate opens. At every step, the selected opening side must be free.

The goal is to find a sequence of pick-and-release actions that places all boxes in their target positions while respecting the gripper capacity and avoiding collisions during release. The code used for the model and the benchmark instances is available at ....

1.1 Parameters and Derived Constants

The proposed MiniZinc model uses five input parameters. The parameter `n_boxes` denotes the total number of boxes in the layer. The parameters `boxes_along_x` and `boxes_along_y` define the grid dimensions of the layer. The parameter `boxes_x_dim` denotes the box dimension parallel to the gripper. In the natural orientation, this corresponds to x; if the layer is rotated, it corresponds to y. Finally, `grip_size` denotes the gripper size S. These parameters are declared as follows:

```
par int: n_boxes;
par int: boxes_x_dim;
par int: boxes_along_x;
par int: boxes_along_y;
par int: grip_size;
```

From these inputs, the model derives three constants. The first one is the maximum number of boxes that can be picked in a single operation, computed as:

$$\texttt{max\_pkble\_boxes} = \left\lfloor \frac{\texttt{grip\_size} + \texttt{boxes\_x\_dim}}{\texttt{boxes\_x\_dim}} \right\rfloor .$$

This value derives directly from the gripper capacity $S + x$. The second constant is the number of groups needed to cover one pallet row:

$$\texttt{groups\_per\_row} = \left\lceil \frac{\texttt{boxes\_along\_x}}{\texttt{max\_pkble\_boxes}} \right\rceil .$$

This is a ceiling division because the last group in a row may contain fewer boxes than the maximum allowed number. The third constant is the total number of groups:

$$\texttt{n\_groups} = \texttt{boxes\_along\_y} \cdot \texttt{groups\_per\_row}.$$

It is obtained by multiplying the number of groups per row by the number of rows. The corresponding MiniZinc declarations are:

```
par int: max_pkble_boxes = (grip_size + boxes_x_dim) div boxes_x_dim;

par int: groups_per_row = (boxes_along_x + max_pkble_boxes - 1)
                          div max_pkble_boxes;

par int: n_groups = boxes_along_y * groups_per_row;
```

2 Grouping

This part of the model admits several possible encodings. A natural but expensive formulation is a *fully enumerated* encoding, where each group explicitly stores the identifiers of all boxes it contains. Since groups may have different lengths, this representation requires a two-dimensional array with padding values for shorter groups.

```
array[1..n_groups, 1..max_pkble_packs] of var 0..n_packs: groups;
```

Here, the value 0 is used as a dummy value to represent an empty slot. This representation makes group membership explicit and easy to inspect. However, it introduces a large number of decision

variables and requires additional constraints to enforce the structure of a valid solution. In particular, the model must ensure that each box appears exactly once, that non-empty positions inside each group are ordered, and that the groups are consistent with the geometric ordering of the layer.

A second possibility is a *binary assignment* encoding, where a boolean variable states whether pack i belongs to group g. This representation can be written as follows:

```
array[1..n_groups, 1..n_packs] of var bool: belongs_to_group;
```

In this case, group membership is represented by the truth value of `belongs_to_group[g,i]`. Although this encoding is expressive, it is even larger than the fully enumerated representation. It also does not encode contiguity or same-row membership directly. These properties must be reconstructed through additional constraints.

For this reason, the adopted model represents each group using its *start index* and *length*. An equivalent formulation could use the start and end indices instead. This interval-style representation is more compact because each group is described by only two main variables. Contiguity is implicit: once the start index and the length are known, the boxes in the group are uniquely determined. The same-row constraint can also be expressed directly by requiring the interval not to cross a row boundary. As a result, the model avoids explicit membership variables and reduces the number of constraints needed to describe valid groups.

```
array[1..n_groups] of var 1..n_packs: group_start;
array[1..n_groups] of var 1..max_pkble_packs: group_len;
```

The adopted encoding uses two arrays: `group_start`, which stores the first box of each group, and `group_len`, which stores the number of boxes in that group.

```
constraint group_start[1] = 1;

constraint forall(g in 1..n_groups - 1)(
  group_start[g + 1] = group_start[g] + group_len[g]
);

constraint group_start[n_groups] + group_len[n_groups] - 1 = n_packs;

constraint forall(g in 1..n_groups)(
  (group_start[g] - 1) div packs_along_x =
  (group_start[g] + group_len[g] - 2) div packs_along_x
);
```

The first three constraints define the groups as consecutive intervals. The first group starts from pack 1, and each following group starts immediately after the last pack of the previous group. In particular, the third constraint forces the last group to end at pack n. Together, these conditions ensure that all packs are covered exactly once, without gaps or overlaps.

The fourth constraint enforces the geometric feasibility of each group: a group cannot cross from one row to the next. The constraint compares the row index of the first pack with the row index of the last pack in the group. If the two indices are equal, the whole interval lies on the same row.

An example of the result of the grouping phase is shown in Figure 2.

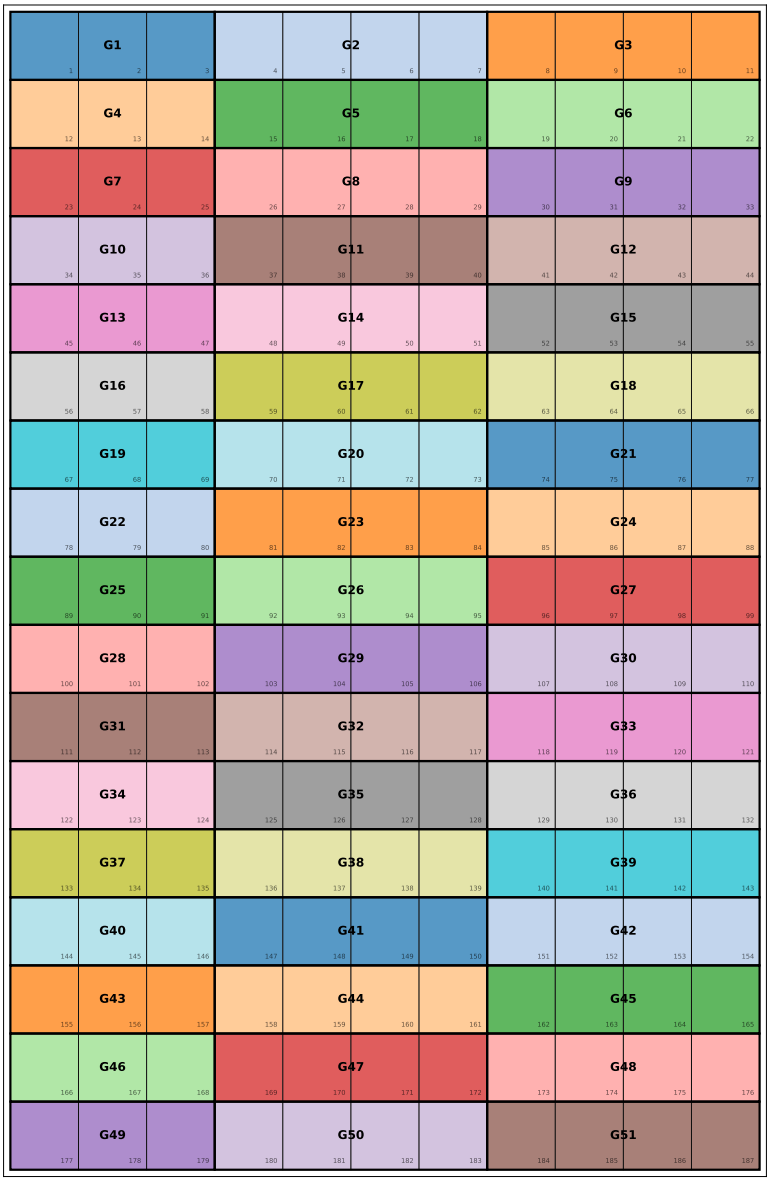


Figure 2: Example pallet layout generated for 70×70 boxes on an 800×1200 pallet. The resulting 11×17 grid contains 187 boxes. The grip size is set to three box widths. Groups alternate 3/4/4 across each row.

3 Scheduling

The second part of the model determines the release order of the groups and selects which gripper plate opens for each release. In this case, the most direct representation consists in assigning to each group a single release position. The array `drop_order` stores the time step at which each group is released, while `plate` stores the plate opened for that release. The value 0 denotes the left plate, and the value 1 denotes the right plate.

```
array[1..n_groups] of var 1..n_groups: drop_order;
array[1..n_groups] of var 0..1: plate;
```

The only global constraint required to make `drop_order` a valid schedule is `alldifferent`.

Since every value in `drop_order` belongs to the domain $1, \ldots,$ `n_groups`, forcing all values to be different ensures that each release step is assigned to exactly one group.

```
constraint alldifferent(drop_order);
```

The feasibility of a release depends on the positions of the adjacent groups in the same vertical column. As shown in Figure 2, groups are numbered from left to right within each row, and then from top to bottom across rows. Therefore, since each row contains `groups_per_row` groups, two groups that are aligned in consecutive rows have indices that differ by `groups_per_row`. For a generic group g, the vertical neighborhood is therefore given by $g \pm$ `groups_per_row`, whenever these indices are within the valid range. Groups in the first pallet row have no upper neighbor, while groups in the last pallet row have no lower neighbor.

When a group is released, the selected plate must open toward a free direction. A direction is free if the group lies on the corresponding pallet boundary, or if the group in that direction is scheduled to be released later and is therefore still absent from the pallet at the current release step.

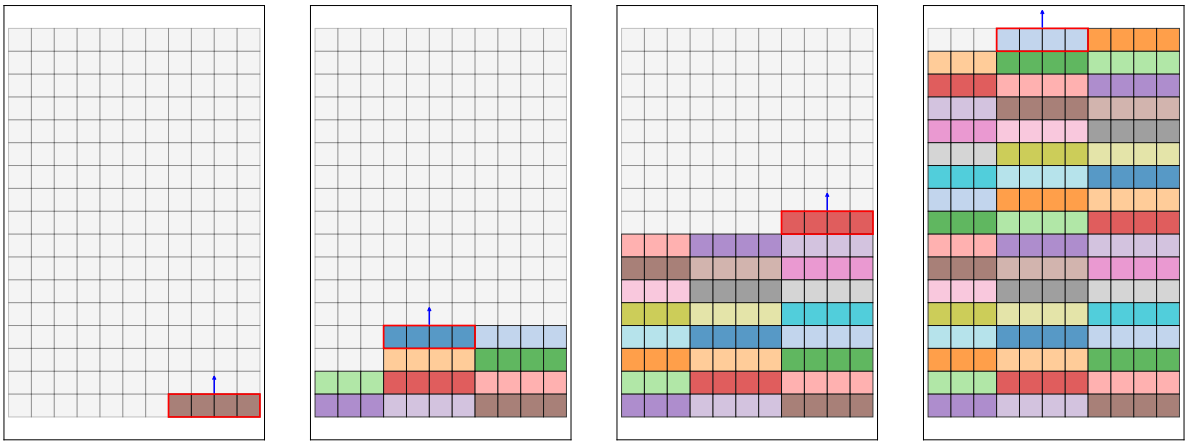
Since the relevant neighbors are above and below the current group, the constraints are written by separating the first row, the internal rows, and the last row. This avoids invalid array accesses at the pallet boundaries.

```
constraint forall(g in 1..n_groups where g <= groups_per_row)(
  plate[g] = 0 \/
  (
    plate[g] = 1 /\
    drop_order[g + groups_per_row] > drop_order[g]
  )
);

constraint forall(g in 1..n_groups where
  g > groups_per_row /\ g <= n_groups - groups_per_row
)(
  (
    plate[g] = 0 /\
    drop_order[g - groups_per_row] > drop_order[g]
  )
  \/
  (
    plate[g] = 1 /\
    drop_order[g + groups_per_row] > drop_order[g]
  )
);

constraint forall(g in 1..n_groups where g > n_groups - groups_per_row)(
  (
    plate[g] = 0 /\
    drop_order[g - groups_per_row] > drop_order[g]
  )
  \/
  plate[g] = 1
);
```

The *first* constraint handles groups in the first pallet row. These groups have no upper neighbor, so opening plate 0 is always feasible. Opening plate 1, instead, is feasible only if the lower neighbor is released later. The *second* constraint handles the internal rows. In this case, both vertical neighbors exist. Opening plate 0 is feasible only if the upper neighbor

$g - \texttt{groups\_per\_row}$ is released later. Similarly, opening plate 1 is feasible only if the lower neighbor $g + \texttt{groups\_per\_row}$ is released later. The *third* constraint handles groups in the last pallet row. These groups have no lower neighbor, so opening plate 1 is always feasible. Opening plate 0 is feasible only if the upper neighbor is released later.

Figure 3 shows an example of such a schedule at different placement steps.



(a) Step 1 (b) Step 10 (c) Step 25 (d) Step 50

Figure 3: Example of a feasible scheduling sequence. The red border marks the group being placed at the selected step, while the blue arrow indicates the opening side of the plate.

4 Solving

The proposed model is solved as a feasibility problem:

```
solve satisfy;
```

No objective function is needed. The goal is only to find one valid schedule. The variables `drop_order` and `plate` define the release order and the opening side for each group. If the vertical-neighbor constraints are satisfied, each release can be performed without collisions. Therefore, any satisfying assignment gives a valid pick-and-place plan.

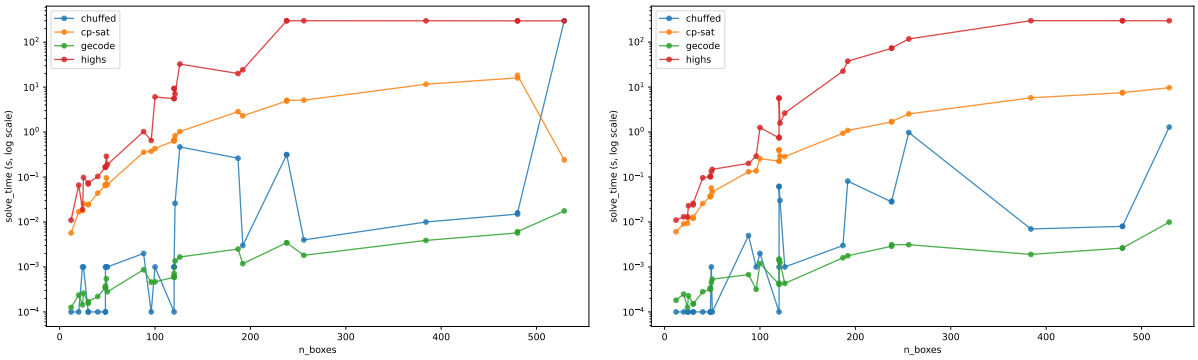
5 Calling the Model from Python

Python is used to run the MiniZinc model on the benchmark instances. The script loads the model and selects the solver:

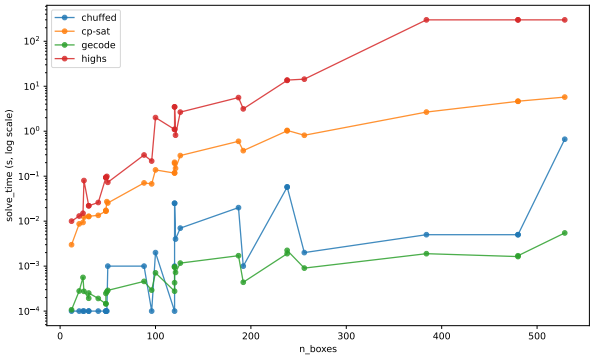
```
model    = Model("model.mzn")
solver   = Solver.lookup("gecode")
instance = Instance(solver, model)
```

It then sets the instance parameters:

```
instance["n_boxes"]       = pallet_info["n_boxes"]
instance["boxes_x_dim"]   = boxes_x_dim
instance["boxes_along_x"] = pallet_info["boxes_along_x"]
instance["boxes_along_y"] = pallet_info["boxes_along_y"]
instance["grip_size"]     = grip_size
```



(a) $k = 1$ (b) $k = 2$



(c) $k = 3$

Figure 4: Runtime scaling for the three gripper multipliers $k \in \{1, 2, 3\}$. The y-axis uses a logarithmic scale.

After the parameters are set, the script calls the solver:

```
result = instance.solve()
```

The output arrays `group_start`, `group_len`, `drop_order`, and `plate` are then read from the `result` map. They give the grouping of the boxes and the release schedule.

6 Benchmarking

The benchmark evaluates the MiniZinc model on every (pallet, pack) combination obtained from the pallet presets in Table 1 and the pack sizes in Table 2.

Pallet preset	Dimensions (mm)
Europallet	1200 × 800
Industrie	1200 × 1000
US 40×48	1219 × 1016
Half-pallet	800 × 600
Australian	1165 × 1165

Table 1: Pallet presets used in the benchmark.

Box size (mm)
50 × 50
70 × 70
100 × 100
120 × 80
150 × 150
200 × 200

Table 2: Box sizes used in the benchmark.

For each combination, the model is tested with four solvers: `gecode`, `cp-sat`, `chuffed`, and

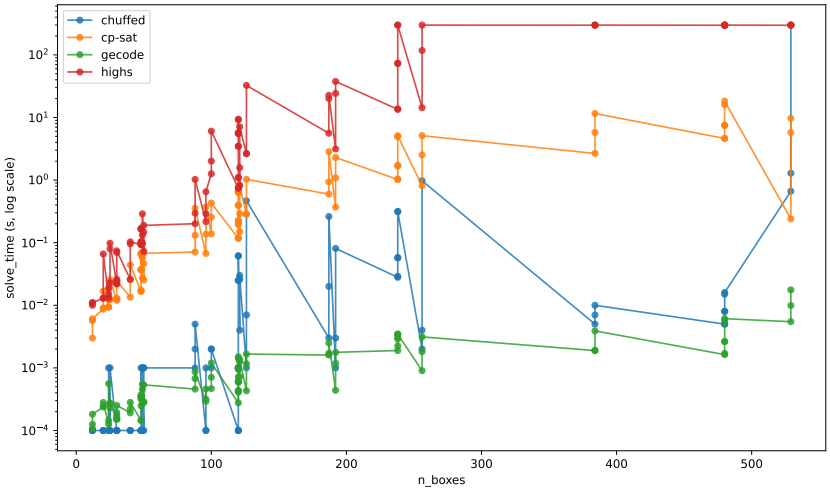


Figure 5: Aggregate runtime scaling over all benchmark instances and solvers. The y-axis uses a logarithmic scale.

`highs`. These solvers represent and solve the problem in different ways. `gecode` and `chuffed` mainly rely on constraint propagation, `cp-sat` uses a SAT-based representation, and `highs` uses a linear programming representation. Each run is executed with a timeout of 300 seconds. The benchmark considers three gripper sizes, $S \in \{1, 2, 3\} \cdot$ `boxes_x_dim`, allowing us to compare how the gripper size affects grouping and scheduling performance.

6.1 Scaling by gripper size

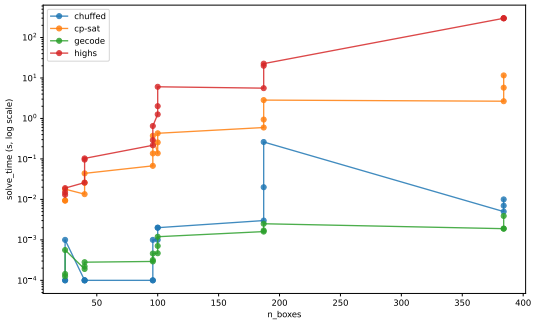
The first comparison fixes the gripper multiplier and measures how the solve time changes with the total number of boxes. Figure 4 shows the resulting scaling curves for $k = 1$, $k = 2$, and $k = 3$. This analysis captures the effect of the gripper size on the proposed model. Larger grippers reduce the number of groups that must be scheduled: the average number of groups decreases from about 75 for $k = 1$ to about 40 for $k = 3$. Therefore, the solve time generally decreases as the multiplier increases.

In particular, `gecode` remains in the lowest runtime range and shows the best scaling behaviour. `cp-sat` shows a clearer dependence on the number of boxes, but remains well below the timeout. `chuffed` is usually fast, although it times out on the densest instance for $k = 1$. `highs` is consistently the slowest solver.

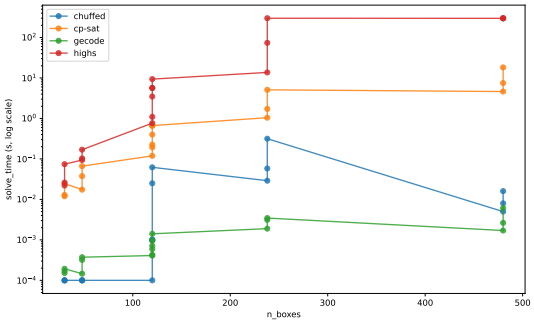
6.2 Overall solver behaviour

After separating the gripper sizes, we aggregate all benchmark runs into a single scaling plot. Figure 5 reports the solve time as a function of the total number of boxes, considering all solvers and all tested instances. This view highlights the global trend across the benchmark and makes the relative behaviour of the solvers easier to compare.

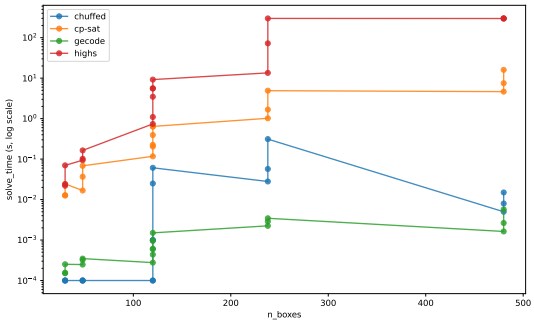
Overall, the comparison confirms that the propagation-based solvers are the most suitable for this model. `gecode` gives the most stable performance and remains close to the bottom of the plot for almost all instances. `chuffed` is also competitive, but its performance is less regular. `cp-sat` shows a stronger dependence on the instance size, although it still remains far from the timeout in most runs. In contrast, `highs` performs worst and several runs approach or reach the timeout.



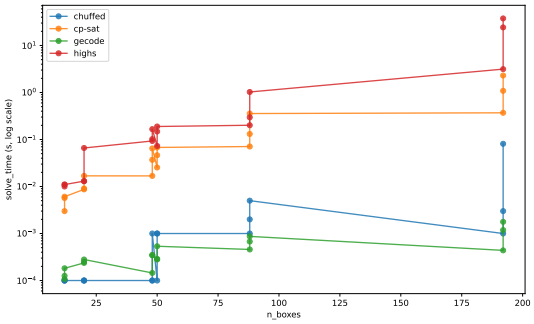
(a) Europallet



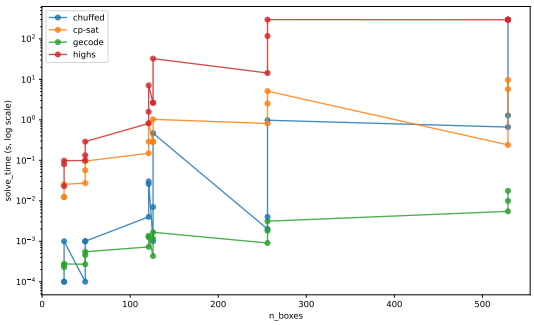
(b) Industrie



(c) US 40×48



(d) Half-pallet



(e) Australian

Figure 6: Runtime comparison across pallet presets. Each subplot aggregates all gripper multipliers for the corresponding preset. The y-axis uses a logarithmic scale.

6.3 Per-pallet breakdown

Another useful comparison studies how the pallet preset influences solver runtime. Differences across presets reflect how `boxes_along_x` and `boxes_along_y` determine the number and arrangement of groups. Figure 6 shows the resulting breakdown.

The relative behaviour of the solvers remains mostly unchanged across pallets: `gecode` stays the most stable, `cp-sat` and `chuffed` remain competitive, and `highs` is usually the slowest. Larger presets, especially the Industrie, US, and Australian pallets, make the separation between solvers more visible because they include denser instances. The half-pallet instances are smaller, so the differences are less pronounced.

6.4 Flattening and execution overhead

The last analysis compares the solver-reported time with the total runtime measured by the Python benchmark script. This comparison estimates the cost of the steps that occur before the

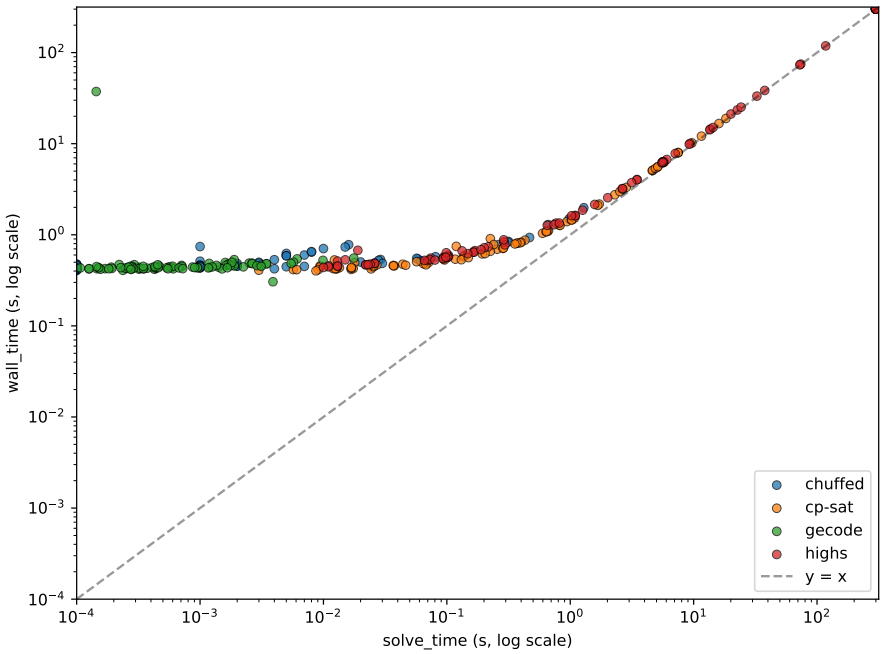


Figure 7: Solver-reported time compared with the total runtime measured by the Python benchmark script.

actual solver search. The main one is MiniZinc flattening: in this phase, the high-level MiniZinc model and the instance data are translated into a lower-level FlatZinc representation, which is then passed to the selected backend solver. The remaining overhead includes Python instance construction, MiniZinc invocation, and process startup.

Figure 7 shows the resulting comparison. The diagonal line marks equal total runtime and solver-reported time. Points close to the diagonal correspond to runs where most of the time is spent inside the solver. Points above the diagonal instead indicate a larger overhead outside the solver; in these cases, flattening and execution costs are more relevant relative to the actual solving time.

The plot shows that this effect is strongest for the fastest runs, especially for `gecode` and several `chuffed` executions. Their solver-reported time is very small, so even a modest overhead produces a visible separation from the diagonal. For slower runs, especially `highs` and the larger `cp-sat` instances, the points are closer to the diagonal. In those cases, the solving phase dominates the total runtime, and the relative overhead becomes less important.

7 Conclusions

The project shows that the robot movement planning problem can be modeled compactly by separating grouping and scheduling decisions. The interval representation of groups, based on start index and length, avoids larger membership-based encodings while still capturing the gripper constraints. The release order is then handled through ordering variables and local neighborhood constraints.

The benchmark shows that the CP-oriented solvers are the most effective on the tested instances. `gecode` is the most stable solver, while `cp-sat` and `chuffed` remain competitive. In contrast, `highs` is consistently slower on this combinatorial formulation. Larger grippers reduce the number of groups and generally improve runtime, while dense layouts with many small boxes remain the hardest cases.